

What we discussed today (so far)

How to assess your current workloads

How to pick the right VM type for your workload

How to size storage for performance, not size

How to assess wait stats in cloud VMs

How to back up directly to cloud-native file systems

Where other cloud database services make sense

Deck 6 p1



Cloud Products Related to SQL Server

Deck 6 p2

Agenda

Azure SQL DB Serverless

AWS Aurora and Babelfish

Azure SQL DB Hyperscale

Azure Synapse Analytics

Azure Arc

Deck 6 p3



Serverless Databases

The idea

You define min & max server sizes:
min & max CPU, memory

The cloud provider watches your workload 24/7

As the workload ramps up,
the cloud provider adds more CPU/memory.

As the workload tapers down,
the cloud provider removes CPU/memory.

Deck 6 p5



Caveats

The new power isn't instantaneous:
think minutes, not seconds

The new power only affects new queries:
once your query starts, it can't get more CPU or RAM

The target audience:

- Very short, easy transactional queries
- Gradually amping up workload
(like users coming into the office)

Deck 6 p6



Who it's not for

Slam on/off loads:

Concert tickets go on sale at exactly 9:00 AM

Reporting servers:

Server sits idle for minutes,

then a big query needs 16 cores and 128GB RAM

Deck 6 p7

The AWS Aurora logo is displayed in white text on a blue rectangular background. The background of the slide is a sunset scene with a layer of white clouds in the foreground.

AWS Aurora

This isn't for SQL Server today.

It's only for MySQL and Postgres.

But I need to explain it so I can teach you
how Azure SQL DB Hyperscale works.

Deck 6 p9



How most databases work

Query Processor	
Storage Engine	
Writes to the transaction log	Reads from the data file

Operating system	
Writes to the transaction log	Reads from the data file

Deck 6 p10



In MySQL & Postgres, all this is open source.

Query Processor	
Storage Engine	
Writes to the transaction log	Reads from the data file

Operating system	
Writes to the transaction log	Reads from the data file

Deck 6 p11



But Amazon controls...

The operating system for Platform-as-a-Service
(like Azure SQL DB)

The storage it writes to

The storage it reads from

Networking between storage in different data centers

Deck 6 p12



So Amazon said

Keep the database front end

Rewrite as little of the database as possible,
just to be aware of the underlying cloud storage

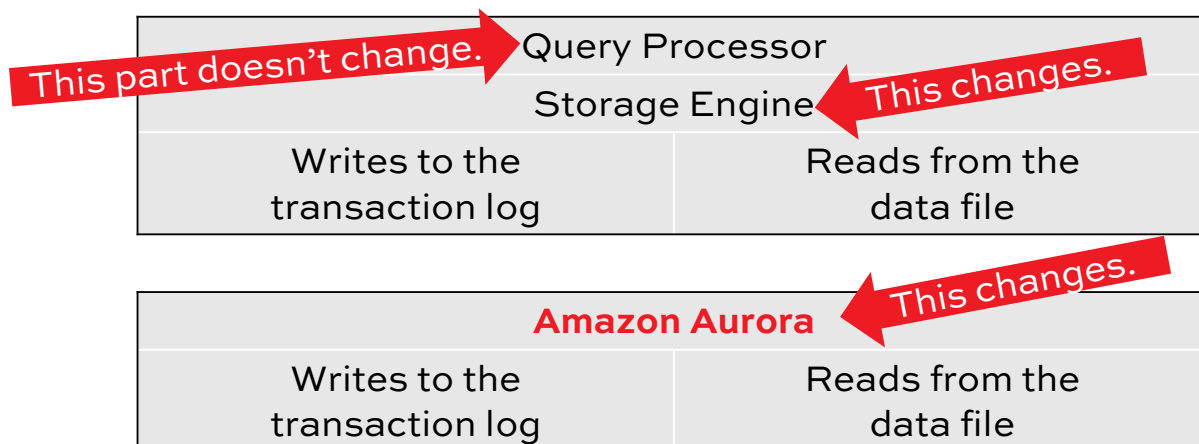
Rewrite the operating system so:

- Writes are committed to multiple data centers
- Failovers can be handled by starting up a new identical database server, anywhere
- Reads can be done on live replicated storage

Deck 6 p13



How they did it



Deck 6 p14



Let's cover these changes first:

Query Processor	
Storage Engine	
Writes to the transaction log	Reads from the data file

Operating system	
Writes to the transaction log	Reads from the data file

Focus here

Deck 6 p15



Amazon Aurora	
Writes to the transaction log	Reads from the data file

DBAs believe:

There is a transaction log “file”

The database server keeps it open & locked

The database server writes to that one file

Deck 6 p16



Amazon Aurora	
 Writes to the transaction log	Reads from the data file

Amazon decided:

There is a transaction log **web service**.

When the database server needs to write to it, the db server sends a message to the log service.

Deck 6 p17



This changes backups, too

When the database server wants to back up the log, the db server sends a message to the log service.

The log service handles the backups.

The db server just keeps right on going.

The db server never touches the log file directly.

Deck 6 p18



So, what happens on crashes?

We're used to thinking:

1. The database server opens the transaction log
2. The database server opens the data file
3. The db server looks at what it was in the middle of doing, then applies those changes to the data files

Hold that thought. The data file is different, too.

Deck 6 p19



DBAs believe:

There is a data “file”

The database server keeps it open & locked

The db server reads data pages from the data file

The db server is responsible for syncing the data & log files

Deck 6 p20





Amazon decided:

There is no data file.

Aurora can build the data pages only when needed: after all, a lot are cached in memory in the db server.

Deck 6 p21



This has interesting effects.

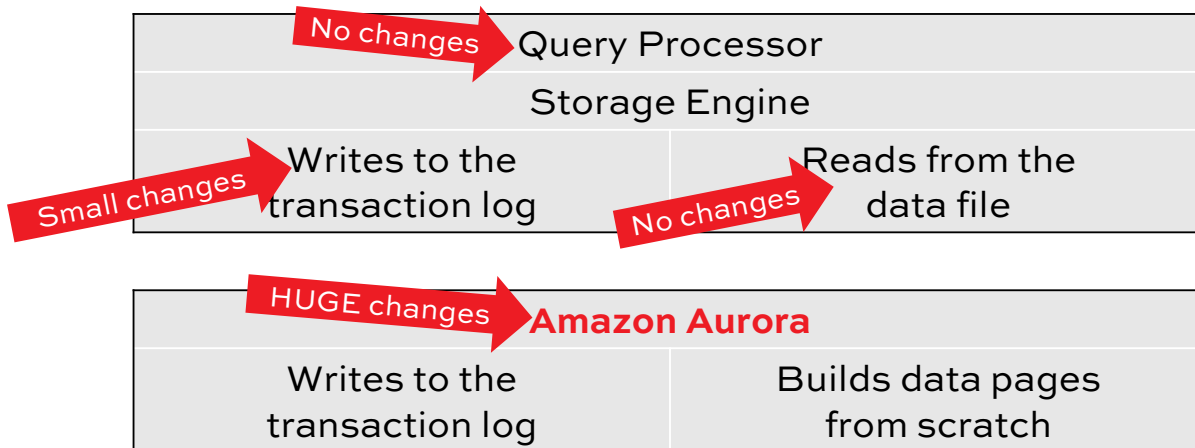
The database server doesn't have to flush data pages to disk anymore, cutting IO.

Crash recovery is effectively done by Aurora, not the database engine.

Deck 6 p22



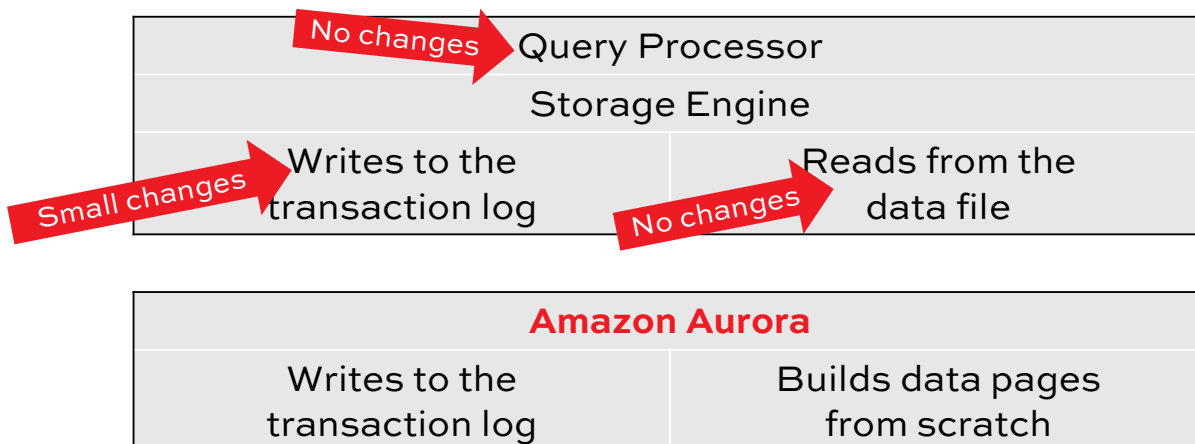
DB changes small, OS changes big.



Deck 6 p23



Now, think about read replicas.



Deck 6 p24



Have as many as you want.

There's no transaction log file to worry about:
no locking, no replication.

Read replicas don't need to:

- Get the log file sent from the primary
- Replay the log transactions to data pages
- The log & data are replicated automatically at the storage layer, not the database layer
- Amazon S3 is great at fast, global replication

Amazon Aurora	
Writes to the transaction log	Builds data pages from scratch

Deck 6 p26

I'm skipping over a LOT.

24-minute internals intro:

<https://www.youtube.com/watch?v=OleLKyBl3CM>

30 more minutes on internals:

<https://www.youtube.com/watch?v=HQg8wqlxefo>

35 minutes on global replication & failover:

<https://www.youtube.com/watch?v=1vFg1z-2E7Y>

Deck 6 p26



SQL ConstantCare® uses it.

Gets data from over 3,000 SQL Servers every day.

2TB database in AWS Aurora Postgres.

100% database turnover every 30 days:
we roll through 2TB of new data every month.

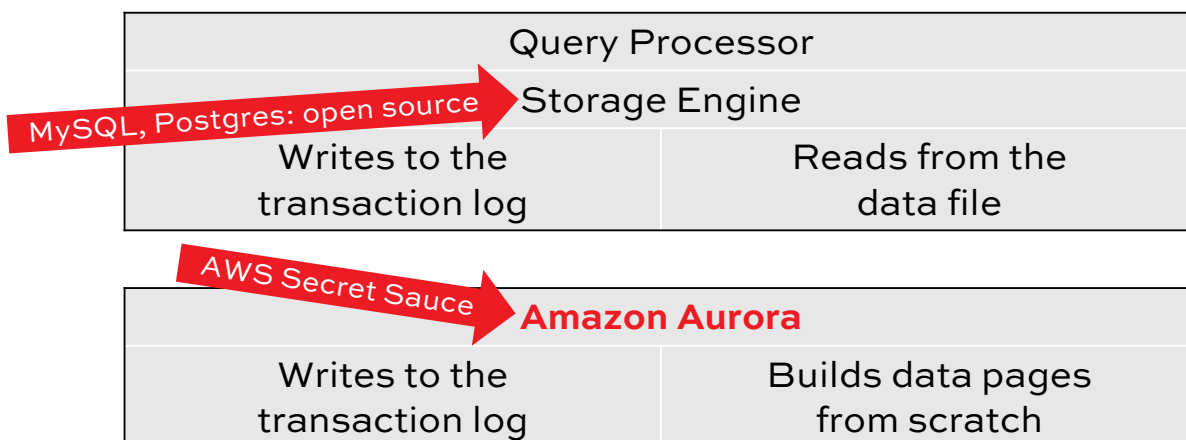
The company is just me & Richie Rump,
and neither of us does database administration on it.

I can see why it's really compelling to developers.

Deck 6 p27



But it's only MySQL and Postgres.



Deck 6 p28



It's not SQL Server, though.

Amazon does have a plan to use it for SQL Server...

But it's not something you're gonna like.

Deck 6 p29

The background of the slide is a blue sky with white, wispy clouds. In the center, there is a solid blue rectangle containing the text "AWS Aurora Babelfish" in white. The text is arranged in two lines: "AWS Aurora" on the top line and "Babelfish" on the bottom line.

AWS Aurora Babelfish

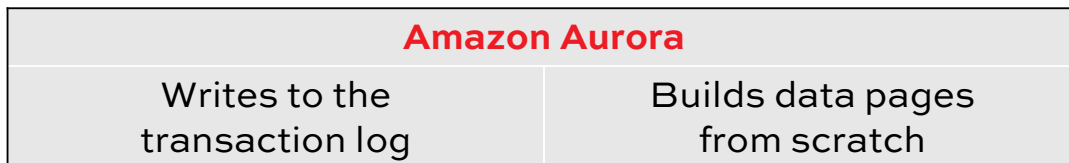
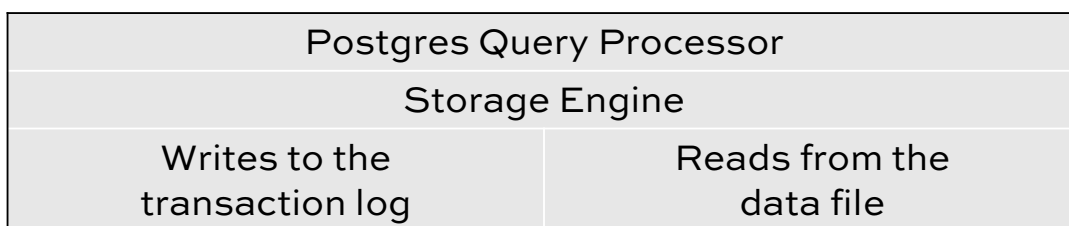
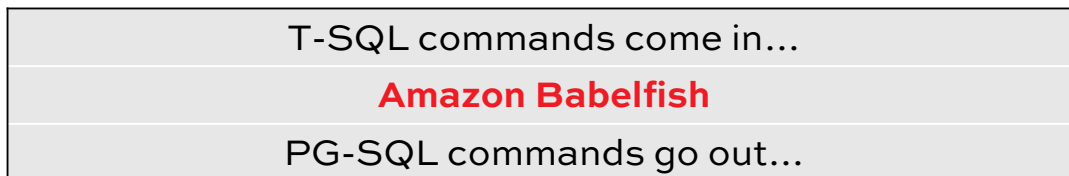
The idea

Your application thinks it's talking to SQL Server

The data lives in a cheap PostgreSQL database

When you connect to SQL Server,
you actually connect to AWS Babelfish:
a translation layer turning T-SQL into PL/pgSQL.

Deck 6 p31



Deck 6 p32



Who it's for

Companies with old legacy SQL Server applications that haven't changed in years (or decades)

Those old apps didn't use cutting-edge SQL Server features introduced in newer versions

The company's decided to move to Postgres

They can't or don't want to rewrite old legacy apps

They don't need any new or edge case SQL features

Deck 6 p33



How those companies can use it

Old legacy app connects to Babelfish, runs queries

New apps connect directly to Postgres

New apps can query legacy tables without having to sync data between SQL Server & Postgres

New apps can leverage newer Postgres features

Deck 6 p34



Not a good fit for apps that...

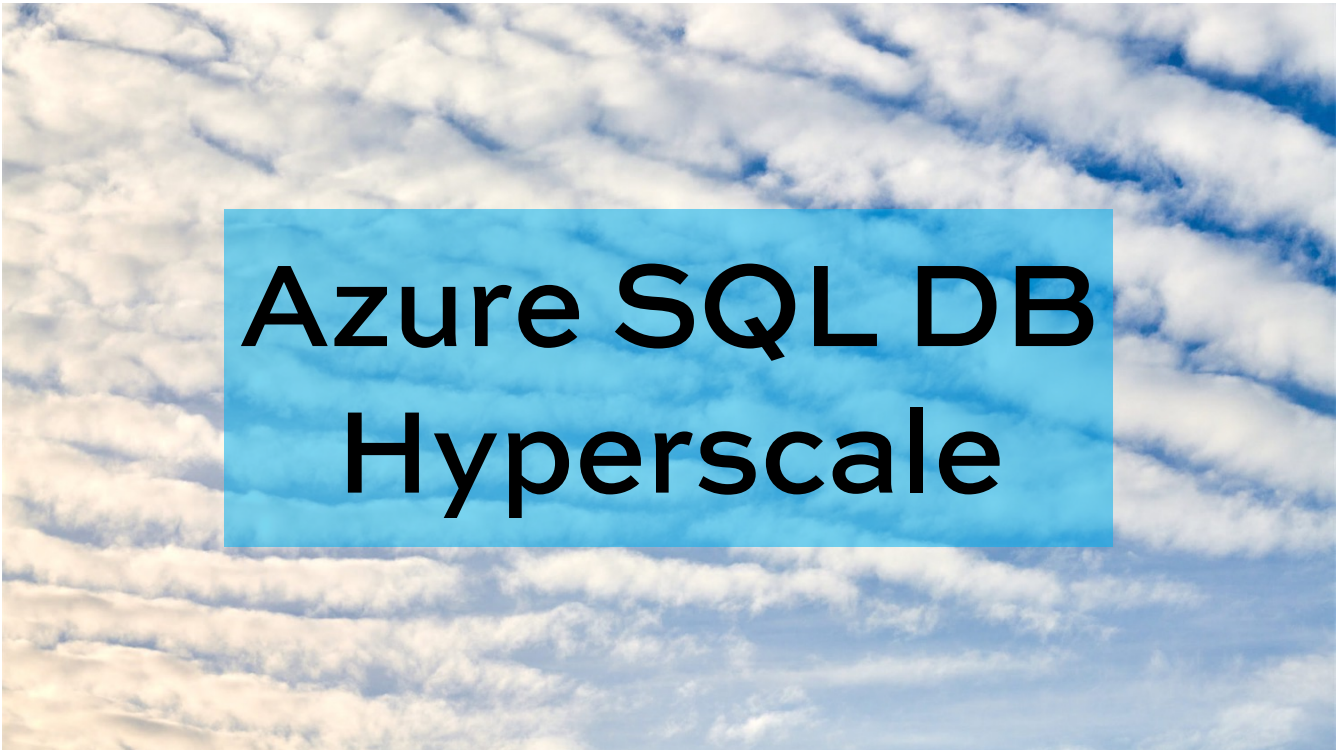
Use edge case database features

Want the new features from new SQL Server versions

Need detailed performance monitoring
(forget DMVs, Query Store, etc)

Let's move on to something you're more likely to use.

Deck 6 p35



**Azure SQL DB
Hyperscale**

Microsoft saw Amazon doing this.

MySQL & Postgres Query Processor	
Storage Engine	
Writes to the transaction log	Reads from the data file

Amazon Aurora	
Writes to the transaction log storage	Builds data pages from scratch

Deck 6 p37



Microsoft: “We can do that too.”

MySQL & Postgres Query Processor	
Storage Engine	
Writes to the transaction log	Reads from the data file

A lot like Aurora

Azure SQL DB Hyperscale

Writes to the transaction log	Builds data pages from scratch
-------------------------------	--------------------------------

Deck 6 p38



Microsoft controlled the db code.

So they could do it all in one product. They took this:

Query Processor	
Storage Engine	
Writes to the transaction log	Reads from the data file

And changed it to...

Deck 6 p39



Microsoft controlled the db code.

So they could do it all in one product. They took this:

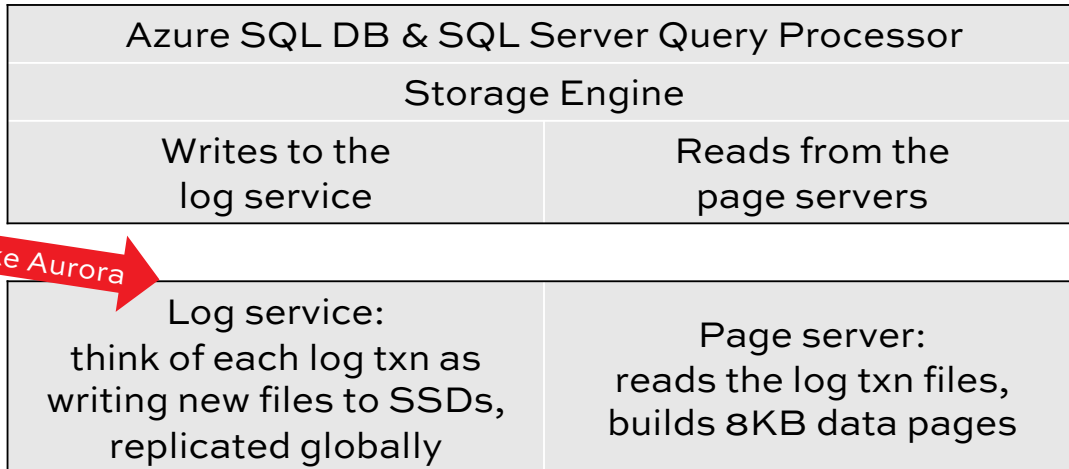
Query Processor	
Storage Engine	
Writes to the log service	Reads from the page servers

To end users, it's just a regular Azure SQL DB.

Deck 6 p40



Microsoft: “We can do that too.”



Deck 6 p41



You can see it in the DMVs.

These products are all kinda sorta the same code:

- Azure SQL DB
- Azure SQL DB Hyperscale
- SQL Server

And they kinda sorta share the same DMVs.

Example: `sys.dm_exec_query_stats` has columns for `page_server_reads`, even in SQL Server.

`Sys.messages` has messages about the page server.

Deck 6 p42





It's for Alex the Architect.

Building a new app on Azure SQL DB

Self-contained in a single database

Expects massive growth, bigger than what's comfortable in Azure SQL DB

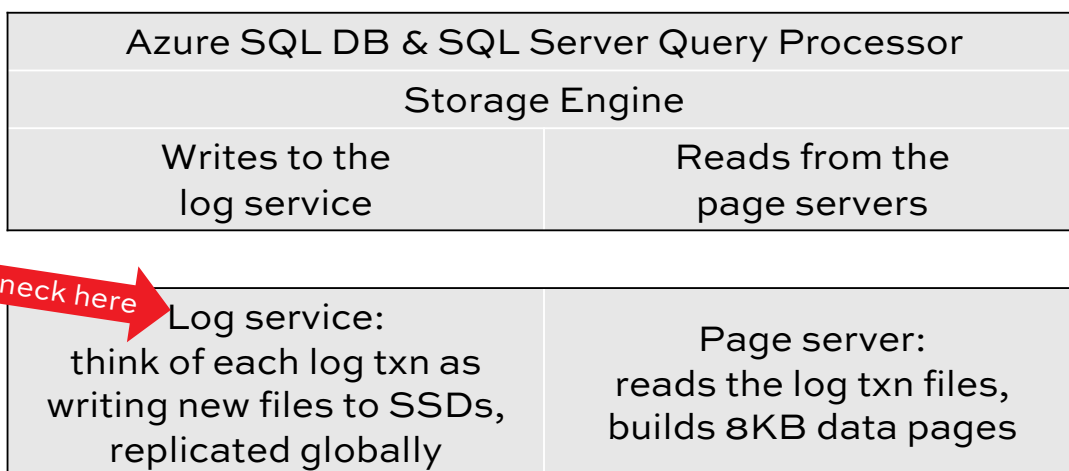
Hyperscale supports up to 100TB databases

Backups, restores, crash recovery, failover all fast because they use snapshots, replication

Deck 6 p43



One major downside...



Deck 6 p44



Log service limit: 100 MB/sec.

If you are running data analytics on a large scale with complex queries and sustained ingestion rates higher than 100 MB/s, or using Parallel Data Warehouse (PDW), Teradata, or other Massively Parallel Processing (MPP) data warehouses, Azure Synapse Analytics may be the best choice.

<https://docs.microsoft.com/en-us/azure/azure-sql/database/service-tier-hyperscale-frequently-asked-questions-faq>

<https://www.brentozar.com/archive/2019/02/how-fast-can-a-5436-mo-azure-sql-db-hyperscale-load-data/>

Deck 6 p45



Learn more about Hyperscale

My explanation with diagrams:

<https://www.brentozar.com/archive/2019/01/how-azure-sql-db-hyperscale-works/>

Summary of a white paper about Socrates:

<http://muratbuffalo.blogspot.com/2022/08/socrates-new-sql-server-in-cloud-sigmod.html>

Deck 6 p46



Azure Synapse Analytics

The history is a mess.

2008: Microsoft buys DATAlegro

2010: Parallel Data Warehouse (PDW)

2012: Hadoop in SQL Server

2014: Analytics Platform System (APS)

2016: Azure SQL Data Warehouse (DW)

2019: SQL Pools, Azure Synapse Analytics,
~~Big Data Clusters~~





The problems with Synapse

	Microsoft	Competition
The problems you're trying to solve	Consistent	Consistent
The solution you built	Keeps changing	Consistent
How you sell the solution	Keeps changing	Consistent

Azure Synapse Analytics' dark past is a hot mess, but let's talk about today.

Just be careful with your career bets.



Problems it's trying to solve

Data warehouse developers can't keep up:

- New data sources keep coming in
- Existing data sources keep changing (app upgrades, new tables & columns)

Nightly load processes can't keep up with raw data.

User query volumes are unpredictable: idle for hours, then massively overloaded with ugly queries.

Deck 6 p51



Microsoft's overall idea

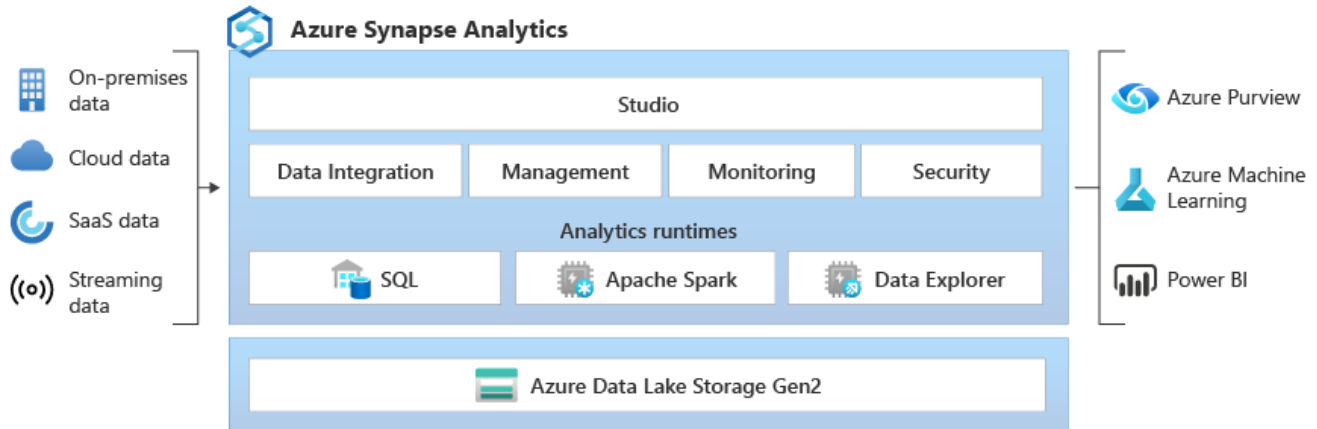
Use one T-SQL query front end to query:

- Stable, normalized data warehouse tables stored in relational databases
- New, crazily changing files (think export files like XML or JSON sitting on a file share)
- One-off data sources, like what the data scientists bring in for a one-off project

Deck 6 p52



The marketing diagram



<https://docs.microsoft.com/en-us/azure/synapse-analytics/overview-what-is>

Deck 6 p53



This isn't a database.

It's a data warehouse
operating system.

Deck 6 p54



Operating system: Windows

You can build apps in a lot of languages
(C#, Java, Python, PowerShell)

You can work with files in a lot of formats
(PDF, Excel, text)

100 Windows users can use it for 100 different things,
but all be supported by 1 sysadmin team.

Deck 6 p55



Operating system: Synapse

You can **work with data** in a lot of languages
(T-SQL, KQL, Spark, Python, PowerBI)

You can work with **data** in a lot of formats
(internal tables, other databases, flat files)

100 **Synapse** users can use it for 100 different things,
but all be supported by 1 **data warehouse team**.

Deck 6 p56



This isn't for relational data.

If your primary need is for a database, you might just need the ability to query flat files, like this:

<https://techcommunity.microsoft.com/t5/azure-sql-blog/data-virtualization-now-generally-available-in-azure-sql-managed/ba-p/3624292>

Data virtualization now generally available in Azure SQL Managed Instance ...

By  Mladen Andzic

Published Sep 12 2022 10:51 AM

👁 1,616 Views

We are excited to announce the general availability (GA) of data virtualization capabilities in Azure SQL Managed Instance, with improved query performance and managed identity as a new supported option for authenticating to storage accounts.

The data virtualization enables you to execute Transact-SQL (T-SQL) queries on files storing data in common data formats in [Azure Data Lake Storage Gen2](#) or [Azure Blob Storage](#) and combine it with relational data stored locally in the managed instance using logical joins. This way you can transparently access external data while keeping it in its original format and location. There is no data duplication or need to run and maintain ETL processes, which means that you can extract and deliver insights faster. The supported file formats are Parquet, CSV, and JSON.



Microsoft has a tough challenge.

Their competition has one product and a vision.

Microsoft has a *lot* of products, with a *lot* of overlap, and they don't publish a long-term vision. So:

	Microsoft	Competition
The problems you're trying to solve	Consistent	Consistent
The solution you built	Keeps changing	Consistent
How you sell the solution	Keeps changing	Consistent

Deck 6 p58



Azure Arc



“We don’t want the cloud.”

- Don’t trust outside providers
- Have legal or compliance issues
- Can run datacenters more cheaply (less reliable hardware, redundancy)
- Can run faster servers with more risk
- Want Azure, but also their own data centers too
- Want Azure, but in locations Microsoft isn’t in
- Want Azure, but also AWS, GCP, China, Russia



Azure Arc: run Azure yourself.

Run Azure services like Azure SQL DB, MI

Microsoft's portal, provisioning tools, accounting

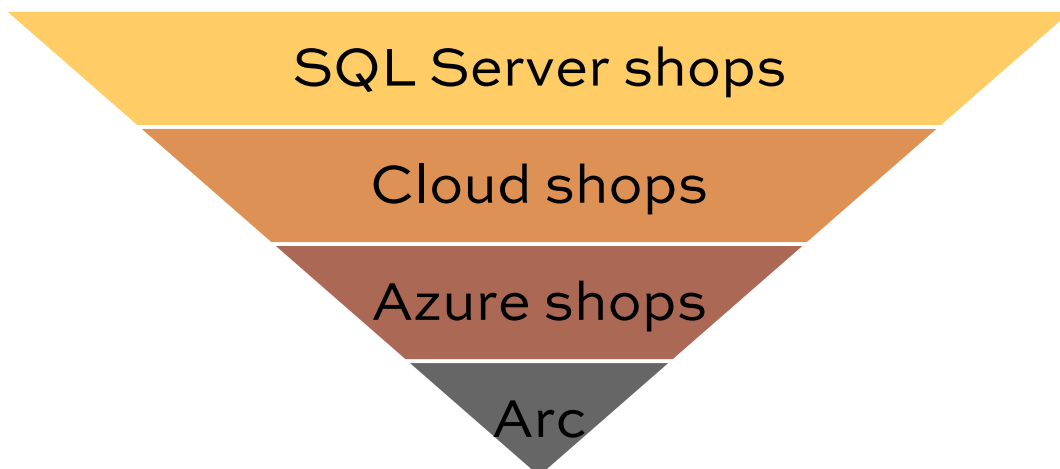
Use your data center, hardware, network, power

Costs money: Microsoft charges you for their tooling

Deck 6 p61



How many shops are there?



Deck 6 p62



Be careful with your career.

You only have so much time to learn stuff.

If you learn something that hardly anybody uses:

- It's going to be really hard to find a job using it
- If it doesn't catch on, you're screwed:
you wasted time learning a dead skill
- Opportunity cost:
you could have learned something valuable

Deck 6 p63



One exception

If you want a job with Microsoft or Amazon:

1. As soon as a super-rare technology is announced, start learning it as quickly as you can
2. Be the only person out there blogging, speaking
3. MS/AWS notices: talk to em, build relationships
4. Get a job with MS/AWS quickly before the tech dies
5. Pivot to a different position inside MS/AWS

Deck 6 p64



Welcome to the cloud.
The learning never stops.
Hopefully you learned valuable stuff today!

Recap



Deck 6 p66



Get these before you plan

1. Total data size in GB/TB, number of databases
2. Batch Requests per second
3. Wait time per core over time (like per second)
4. Is the same performance a success or a failure?
5. How much growth do we expect this year?
6. Can we restrain ourselves to Azure SQL DB features?

These predict the ease of cloud migrations.

Deck 6 p67



Cloud deployment options

Platform-as-a-Service (Azure SQL DB, Amazon RDS) is best for developer shops who don't want DBAs.

- Alex the Architect defaults to this.
- Billie's Business should use it wherever possible.

Infrastructure-as-a-Service (Amazon EC2, Azure VM) is like building blocks, and you have new techniques.

- Elliot's Enterprise prefers this.

Both keep changing really, really quickly.

Deck 6 p68



Cloud = 2000s-era storage.

Storage hardware is hidden in plan read operators

From 2000s to 2020s,
RAM & storage improved 20X-1000X

Cloud hardware regressed back to the 2000s

To handle it, we have to use 2000s-era techniques:

- Get pools of drives for speed, not 1 for capacity
- Pick your VM instance type based on throughput

Deck 6 p69



Design backups first.

For a successful cloud migration:

1. Design how you want to back up SQL Server.
2. If your solution requires native backup & restore, not using EBS/Blob snapshots, provision cloud VMs to test backup/restore speed.
3. You'll often find the backup/restore speed dictates the minimum VM instance type & size you can use.

Deck 6 p70



Design backups first, then...

As long as backups & restores meet your RPO/RTO:

- You've got the right instance type
(to get enough storage throughput)
- You just may not have the right size
(to get the query performance you want)

Don't overthink that: it's easy to change size later.

Go live, then measure performance & adjust sizes.

Deck 6 p71



Go live a little oversized.

When you lift-and-shift to the cloud,
start with the **right instance type**
based on your backup/restore time goals.

But use a **slightly bigger instance size**
with more CPU & RAM than you think you need.

Back it down over time as you measure waits.

Focus on fixing your top wait type's root cause.

Deck 6 p72



**Thanks,
and I hope you
had a great time!**

Deck 6 p73

